

פרויקט: CRUD לניהול צורות עם OOP ו-JSON

מטרת הפרויקט

לבנות מערכת שמנהלת רשימת צורות גיאומטריות.

המערכת תאפשר:

- Create - הוספת צורה חדשה
- Read - הצגת כל הצורות
- Update - עדכון צורה קיימת
- Delete - מחיקת צורה

הנתונים יישמרו בקובץ JSON, כך שגם אחרי סגירת התוכנית הנתונים יישארו.

מבנה תיקיות

```
shape_crud/
|
|— main.py
|— shape.py
|— rectangle.py
|— square.py
|— circle.py
|— shape_manager.py
|— shapes.json
```

הרעיון הכללי

במקום רק לחשב שטח והיקף, עכשיו ננהל מאגר של צורות.

כל צורה תישמר כ-object בקוד, אבל בקובץ היא תישמר כ-JSON.

לדוגמה:

```
[
{
  "id": 1,
```

```
"type": "square",  
"side": 5  
,  
{  
  "id": 2,  
  "type": "circle",  
  "radius": 3  
}  
]
```

שלב 1: מחלקת בסיס Shape

קובץ:

shape.py

המחלקה תכיל:

```
class Shape:  
    def __init__(self, shape_id, shape_type):  
        self.id = shape_id  
        self.shape_type = shape_type  
  
    def get_area(self):  
        pass  
  
    def get_perimeter(self):  
        pass  
  
    def to_dict(self):  
        pass
```

שלב 2: מחלקות צורות

לדוגמה **Shape**:

```
class Square(Shape):  
  
    }
```

אותו רעיון עבור:

Rectangle
Circle
Triangle
Hexagon

שלב 3: ShapeManager

קובץ:

shape_manager.py

זו המחלקה שתנהל את כל פעולות ה-CRUD.

היא תכיל:

```
class ShapeManager:  
    def __init__(self):  
        self.shapes = []  
        self.load_from_json()  
  
    def create_shape(self, shape):  
        pass  
  
    def get_all_shapes(self):  
        pass  
  
    def update_shape(self, shape_id, new_data):  
        pass  
  
    def delete_shape(self, shape_id):  
        pass  
  
    def save_to_json(self):  
        pass
```

```
def load_from_json(self):  
    pass
```

שלב 4: פעולת Create

הוספת צורה חדשה לרשימה ושמירה ל-JSON.

לדוגמה:

בחר צורה:

1. Square

2. Rectangle

3. Circle

המשתמש יכניס נתונים, ואז המערכת תיצור object ותשמור אותו.

שלב 5: פעולת Read

הצגת כל הצורות הקיימות.

פלט לדוגמה:

ID: 1
Type: Square
Side: 5
Area: 25
Perimeter: 20

שלב 6: פעולת Update

המשתמש יכניס ID של צורה.

המערכת תמצא את הצורה ותאפשר לעדכן את הערכים שלה.

לדוגמה:

Enter shape id: 1
Enter new side: 10

לאחר העדכון שומרים שוב ל-JSON.

שלב 7: פעולת Delete

המשתמש יכניס ID.

אם הצורה קיימת, מוחקים אותה מהרשימה ושומרים מחדש ל-JSON.

Shape deleted successfully

שלב 8: main.py

קובץ ראשי עם תפריט:

- 1. Add shape
- 2. Show all shapes
- 3. Update shape
- 4. Delete shape
- 5. Exit

דרישות חובה

- 1. שימוש ב-OOP
- 2. כל מחלקה בקובץ נפרד
- 3. שמירה לקובץ JSON
- 4. טעינה מקובץ JSON

- 5. CRUD מלא
- 6. טיפול בשגיאות
- 7. תפריט ראשי
- 8. שימוש ב-Git

הרחבה למי שסיים את הכל

אפשר להוסיף:

- חיפוש צורה לפי ID
- חיפוש לפי סוג צורה
- מיון לפי שטח
- השוואה בין שתי צורות
- מחיקת כל הצורות